

Tecnatura Superior en Desarrollo de Software

Módulo 07 — InterfaceAdapter: Presenters

Actividad Práctica — CU-CLI-001: Alta de Cliente | Asignatura: Programación II | Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat | Ciclo Lectivo 2026

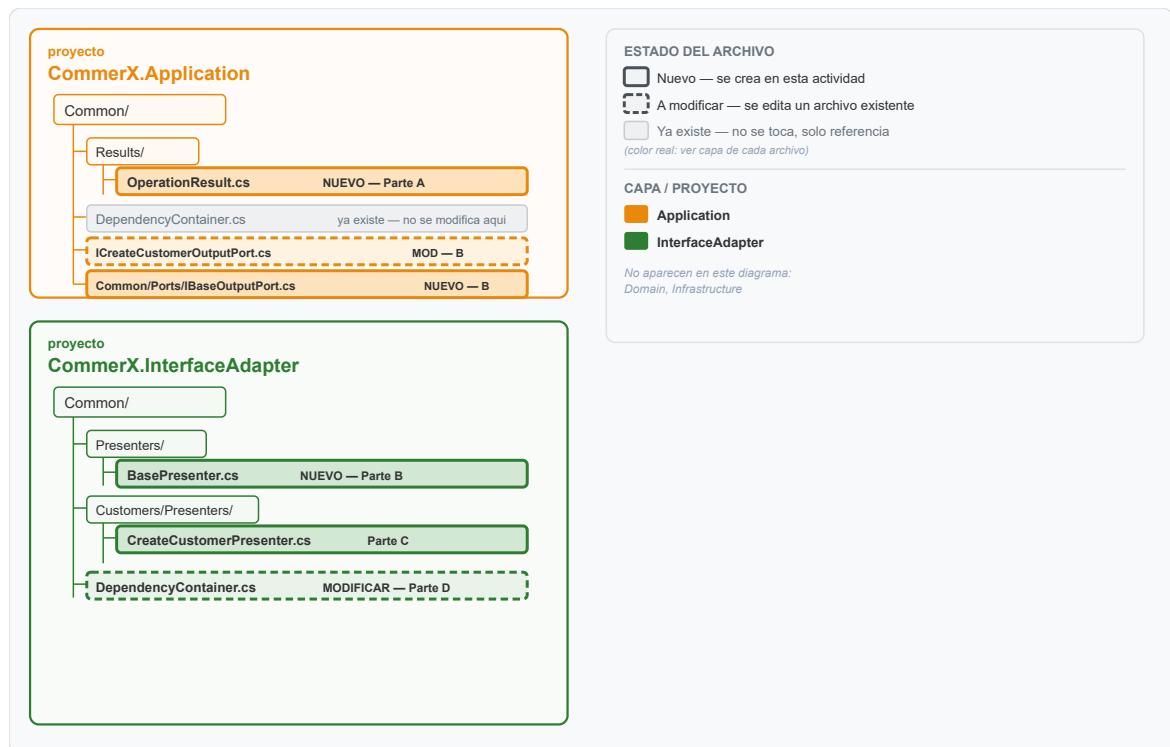
Contexto de la actividad

En el **Módulo 06** completaste la capa Application del caso de uso *Alta de Cliente*: el UseCase valida el request a través del `CreateCustomerValidatorHub`, crea la entidad, persiste y notifica el resultado a través de `ICreateCustomerOutputPort`.

El `OutputPort`, sin embargo, es solo una interfaz — nadie la implementa todavía. Y tiene una limitación: `ICreateCustomerOutputPort` quedó en Módulo 04 como una interfaz plana de cinco métodos, sin ningún contrato compartido con los `OutputPorts` de otros casos de uso (por ejemplo, `IUpdateCustomerOutputPort`). En esta actividad construí la capa **InterfaceAdapter** — la clase concreta que recibe esas notificaciones y las almacena en un `OperationResult<T>` — y de paso **refactorizás** ese `OutputPort` plano hacia una jerarquía reutilizable, tal como se explica en el [Anexo 07B](#).

Objetivo: Crear `OperationResult<T>` en Application/Common, crear `IBaseOutputPort<T>` en Application/Common y **modificar** `ICreateCustomerOutputPort` para que lo extienda, crear `BasePresenter<T>` en InterfaceAdapter, implementar `CreateCustomerPresenter`, y registrar todo en el `DependencyContainer` de InterfaceAdapter.

Archivos que vas a crear



Parte A — `OperationResult<T>`

Antes de construir el Presenter, necesitas la clase que va a acumular el resultado de cada notificación del OutputPort. `OperationResult<T>` vive en la capa Application (es un contrato de aplicación, no de UI) y se crea en `Common/Results/`.

Archivo: `CommerX.Application/Common/Results/OperationResult.cs`

```

1 namespace CommerX.Application.Common.Results;
2
3 // clase sellada — no puede heredarse; solo se crea con Ok() o Fail()
4 public sealed class OperationResult<T>
5 {
6     // constructor privado — garantiza que no exista estado indeterminado
7     private OperationResult(bool isSuccess, T? value, IReadOnlyList<string> error
8     s)
9     {
10         IsSuccess = isSuccess;

```

```

10         Value     = value;
11         Errors     = errors;
12     }
13
14     // true si la operación terminó con éxito
15     public bool IsSuccess { get; }
16
17     // dato producido – válido solo cuando IsSuccess es true
18     public T? Value { get; }
19
20     // mensajes de error – válidos solo cuando IsSuccess es false
21     public IReadOnlyList<string> Errors { get; }
22
23     // crea un resultado exitoso con el valor producido
24     public static OperationResult<T> Ok(T value)
25         => new(true, value, Array.Empty<string>());
26
27     // crea un resultado fallido con lista de mensajes
28     public static OperationResult<T> Fail(IReadOnlyList<string> errors)
29         => new(false, default, errors);
30
31     // sobrecarga para un único mensaje de error
32     public static OperationResult<T> Fail(string error)
33         => new(false, default, new[] { error });
34 }

```

Ubicación en la arquitectura: `OperationResult<T>` vive en `Application`, no en `InterfaceAdapter`, porque es el contrato que el `Presenter` necesita. `InterfaceAdapter` depende de `Application` — no al revés.

Parte B — `IBaseOutputPort<T>` y refactor de `ICreateCustomerOutputPort`

En Módulo 04 creaste `ICreateCustomerOutputPort` como una interfaz **plana**: sus cinco métodos —éxito, dos duplicados, error de dominio y errores de validación— viven todos ahí, sin ningún contrato compartido con el `OutputPort` de otro caso de uso (por ejemplo `IUpdateCustomerOutputPort`). Antes de construir el `Presenter` vas a extraer los tres canales que **cualquier** `OutputPort` del sistema necesita hacia una interfaz base — `IBaseOutputPort<T>` — y vas a **modificar** `ICreateCustomerOutputPort` para que la extienda. El

razonamiento completo, con la analogía del hospital y los errores comunes del patrón, está desarrollado en el [Anexo 07B](#); acá vas directo a la implementación sobre CU-CLI-001.

Paso 1 — Crear `IBaseOutputPort<T>`

Archivo (nuevo): `CommerX.Application/Common/Ports/IBaseOutputPort.cs`

```
1 using CommerX.Application.Common.Validation;
2
3 namespace CommerX.Application.Common.Ports;
4
5 // contrato universal – todo OutputPort del sistema expone estos 3 canales
6 public interface IBaseOutputPort<T>
7 {
8     // canal 1 – la operación se completó con éxito
9     Task HandleSuccessAsync(T response);
10
11     // canal 2 – el DTO no pasó las precondiciones técnicas del ValidatorHub
12     Task ValidationErrorsAsync(IReadOnlyList<ValidationError> errors);
13
14     // canal 3 – una regla de dominio fue violada (DomainException capturada)
15     Task HandleErrorAsync(string message);
16 }
```

Paso 2 — Modificar `ICreateCustomerOutputPort`

Compará el "antes" (lo que ya tenés de Módulo 04) con el "después" que necesitás ahora:

Antes — Módulo 04, interfaz plana (5 métodos):

```
1 public interface ICreateCustomerOutputPort
2 {
3     Task HandleSuccessAsync(CreateCustomerResponse response);
4     Task HandleDuplicateDocumentAsync(string document);
5     Task HandleDuplicateEmailAsync(string email);
6     Task HandleValidationErrorAsync(string message);
7     Task ValidationErrorsAsync(IReadOnlyList<ValidationError> errors);
8 }
```

Después — extiende la base, solo 2 métodos propios:

```

1  using CommerX.Application.Common.Ports;
2  using CommerX.Application.Customers.DTOs;
3
4  namespace CommerX.Application.Customers.Ports;
5
6  // extiende los 3 canales universales – agrega solo lo propio de Alta de Cliente
7  public interface ICreateCustomerOutputPort : IBaseOutputPort<CreateCustomerResponse>
8  {
9      Task HandleDuplicateDocumentAsync(string document);
10     Task HandleDuplicateEmailAsync(string email);
11 }

```

¿Qué cambió exactamente?

`HandleSuccessAsync` y `ValidationErrorsAsync` ya no se declaran acá — los hereda de `IBaseOutputPort<CreateCustomerResponse>`. `HandleValidationErrorAsync(string message)` se reemplaza por el canal base `HandleErrorAsync(string message)` — mismo propósito (una `DomainException` capturada), nombre unificado para todo el sistema. Los únicos métodos que quedan en la interfaz específica son los que **ningún otro** `OutputPort` tiene: `HandleDuplicateDocumentAsync` y `HandleDuplicateEmailAsync`.

Parte C — `BasePresenter<T>`

`BasePresenter<T>` es la clase abstracta que centraliza el comportamiento común a todos los presenters de `InterfaceAdapter`. Implementa los tres canales de `IBaseOutputPort<T>` que acabás de crear en la Parte B, y almacena el resultado en el campo protegido `_result`.

Los dos métodos específicos de `Customer` — `HandleDuplicateDocumentAsync` y `HandleDuplicateEmailAsync` — no van acá: son propios de este `UseCase` y se implementan en el `Presenter` concreto (Parte D).

Archivo: `CommerX.InterfaceAdapter/Common/Presenters/BasePresenter.cs`

```

1  using CommerX.Application.Common.Ports;

```

```
2 using CommerX.Application.Common.Results;
3 using CommerX.Application.Common.Validation;
4
5 namespace CommerX.InterfaceAdapter.Common.Presenters;
6
7 // clase abstracta – implementa los 3 canales de IBaseOutputPort<T> que cualquier
  Presenter hereda
8 public abstract class BasePresenter<T> : IBaseOutputPort<T>
9 {
10     // campo protegido – las subclases escriben aquí el resultado
11     protected OperationResult<T>? _result;
12
13     // propiedad pública – el ViewModel la lee después del ExecuteAsync
14     public OperationResult<T>? Result => _result;
15
16     // canal 1 – notificación de éxito – almacena el DTO de respuesta
17     public virtual Task HandleSuccessAsync(T response)
18     {
19         _result = OperationResult<T>.Ok(response);
20         return Task.CompletedTask;
21     }
22
23     // canal 2 – notificación de errores de validación del DTO – lista de mensajes
24     public virtual Task ValidationErrorsAsync(IReadOnlyList<ValidationError> errors)
25     {
26         // extrae los mensajes de cada ValidationError y construye el resultado fallido
27         _result = OperationResult<T>.Fail(
28             errors.Select(e => e.ErrorMessage).ToList());
29         return Task.CompletedTask;
30     }
31
32     // canal 3 – notificación de una regla de dominio violada (DomainException capturada)
33     public virtual Task HandleErrorAsync(string message)
34     {
35         _result = OperationResult<T>.Fail(message);
36         return Task.CompletedTask;
37     }
38 }
```

¿Por qué virtual? Los métodos son `virtual` para que una subclase pueda sobrescribirlos si necesita comportamiento adicional — por ejemplo, actualizar una propiedad de la UI además de guardar el resultado. Por defecto, el comportamiento base alcanza para la mayoría de los casos.

Parte D — CreateCustomerPresenter

`CreateCustomerPresenter` es la clase concreta que implementa `ICreateCustomerOutputPort` y hereda de `BasePresenter<CreateCustomerResponse>`.

La base ya implementó los tres canales de `IBaseOutputPort<T>`: `HandleSuccessAsync`, `ValidationErrorsAsync` y `HandleErrorAsync`. En esta clase solo debés implementar los dos métodos específicos que agregaste a `ICreateCustomerOutputPort` en la Parte B: `HandleDuplicateDocumentAsync` y `HandleDuplicateEmailAsync`.

Archivo: `CommerX.InterfaceAdapter/Customers/Presenters/CreateCustomerPresenter.cs`

```
1 using CommerX.Application.Common.Results;
2 using CommerX.Application.Customers.DTOs;
3 using CommerX.Application.Customers.Ports;
4 using CommerX.InterfaceAdapter.Common.Presenters;
5
6 namespace CommerX.InterfaceAdapter.Customers.Presenters;
7
8 // clase sellada – implementa el OutputPort concreto del caso de uso Alta de Cliente
9 public sealed class CreateCustomerPresenter
10     : BasePresenter<CreateCustomerResponse>,
11       ICreateCustomerOutputPort
12 {
13     // propiedad del contrato – expone el DTO de respuesta directamente
14     public CreateCustomerResponse Response
15         => _result?.Value ?? default!;
16
17     // cliente duplicado – el documento ya existe en el sistema
18     public Task HandleDuplicateDocumentAsync(string document)
19     {
20         // construye un resultado fallido con mensaje de documento duplicado
```

```
21         _result = OperationResult<CreateCustomerResponse>.Fail(  
22             $"El documento {document} ya está registrado.");  
23         return Task.CompletedTask;  
24     }  
25  
26     // cliente duplicado – el email ya existe en el sistema  
27     public Task HandleDuplicateEmailAsync(string email)  
28     {  
29         // construye un resultado fallido con mensaje de email duplicado  
30         _result = OperationResult<CreateCustomerResponse>.Fail(  
31             $"El email {email} ya está registrado.");  
32         return Task.CompletedTask;  
33     }  
34  
35     // HandleSuccessAsync, ValidationErrorsAsync y HandleErrorAsync se heredan de  
36     BasePresenter
```

¿Qué hereda de la base? `HandleSuccessAsync`, `ValidationErrorsAsync` y `HandleErrorAsync` ya están implementados en `BasePresenter<T>`. El Presenter concreto *solo escribe los métodos que le son propios*: `HandleDuplicateDocumentAsync` y `HandleDuplicateEmailAsync`.

Parte E — `DependencyContainer.cs`

El `DependencyContainer` de `InterfaceAdapter` registra el Presenter como `Scoped` y además registra el `OutputPort` apuntando a la misma instancia del Presenter mediante una factory. Esto garantiza que el UseCase y el ViewModel compartan la misma instancia dentro del mismo request.

Archivo: `CommerX.InterfaceAdapter/DependencyContainer.cs`

```
1     using CommerX.Application.Customers.Ports;  
2     using CommerX.InterfaceAdapter.Customers.Presenters;  
3  
4     // namespace truco – extiende IServiceCollection sin agregar using en Program.cs  
5     namespace Microsoft.Extensions.DependencyInjection;  
6  
7     public static class DependencyContainer  
8     {  
9         public static IServiceCollection AddCustomerPresenter(  
10
```



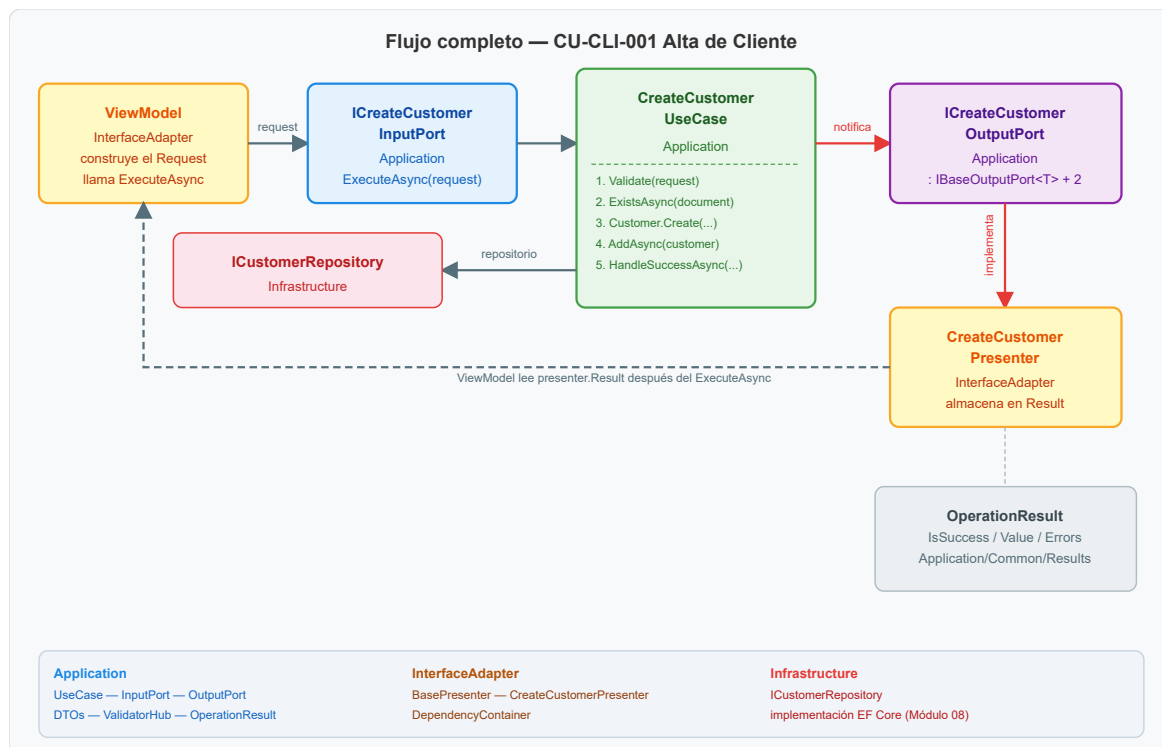
```
10         this IServiceCollection services)
11     {
12         // registrar el Presenter como Scoped – una instancia por request
13         services.AddScoped<CreateCustomerPresenter>();
14
15         // registrar el OutputPort como alias de la misma instancia del Presenter
16         services.AddScoped<ICreateCustomerOutputPort>()
17             .AddScoped(sp => sp.GetRequiredService<CreateCustomerPresenter>());
18
19         return services;
20     }
21 }
```

¿Por qué la factory con GetRequiredService?

Cuando el contenedor DI resuelve `ICreateCustomerOutputPort`, en lugar de crear una nueva instancia llama a la factory que recupera el `CreateCustomerPresenter` ya registrado en el mismo scope. De esta forma el UseCase (que recibe el OutputPort) y el ViewModel (que recibe el Presenter directamente) trabajan sobre el mismo objeto.

Parte F — Flujo visual del caso de uso completo

El siguiente diagrama muestra cómo se conectan todas las piezas que construiste a lo largo de los módulos 04 a 07:



Parte G — Checklist de verificación

Antes de continuar, verificá que cada ítem esté completo:

Archivo	Verificación
OperationResult.cs	Constructor privado, propiedades IsSuccess / Value / Errors, métodos Ok() y Fail() (2 sobrecargas)
IBaseOutputPort.cs (nuevo)	Interfaz genérica con 3 canales: HandleSuccessAsync(T), ValidationErrorsAsync(...), HandleErrorAsync(string)
ICreateCustomerOutputPort.cs (modificado)	Extiende IBaseOutputPort<CreateCustomerResponse>; solo declara HandleDuplicateDocumentAsync y HandleDuplicateEmailAsync
BasePresenter.cs	Implementa IBaseOutputPort<T>; campo protegido _result, propiedad Result, métodos virtual HandleSuccessAsync, ValidationErrorsAsync y HandleErrorAsync

Archivo	Verificación
CreateCustomerPresenter.cs	Hereda BasePresenter<CreateCustomerResponse>, implementa ICreateCustomerOutputPort, propiedad Response, HandleDuplicateDocumentAsync y HandleDuplicateEmailAsync (nada más — el resto se hereda)
DependencyContainer.cs	Namespace truco, AddCustomerPresenter, AddScoped para Presenter, AddScoped para OutputPort con factory GetRequiredService

Parte H — Preguntas de reflexión

1. ¿Por qué el UseCase recibe `ICreateCustomerOutputPort` en lugar de recibir `CreateCustomerPresenter` directamente?
Reflexioná sobre la dirección de dependencias en Clean Architecture.
2. ¿Qué problema produce registrar el Presenter como `Transient`?
Trazá mentalmente el flujo: UseCase escribe en la instancia A, ViewModel lee de la instancia B.
3. ¿Para qué sirve la propiedad `Result` en `BasePresenter<T>`?
¿Quién la lee y cuándo?
4. Si mañana el sistema necesita notificar el resultado por email además de mostrarlo en pantalla, ¿qué cambiaría y qué no?
Pensá en la apertura a la extensión sin modificar el UseCase.
5. ¿En qué difieren `HandleErrorAsync` y `ValidationErrorsAsync` del `OutputPort`?
Revisá el `IBaseOutputPort<T>` y el `CreateCustomerUseCase` para entender cuándo se llama cada uno.
6. ¿Por qué `OperationResult<T>` vive en `Application` y no en `InterfaceAdapter`?
Pensá en quién lo produce y quién lo consume.



7. ¿Qué hubiera pasado si en vez de crear `IBaseOutputPort<T>` dejábamos `ICreateCustomerOutputPort` como en Módulo 04?

Pensá en el día en que el sistema tenga veinte casos de uso, cada uno con su propio `HandleSuccessAsync` y su propio `ValidationErrorsAsync` repetidos letra por letra.

Módulo 07 — Capa InterfaceAdapter: Presenters y ViewModels | Actividad Práctica — CU-CLI-001 | Programación II | IES
N.º 9007 Dr. Salvador Calafat | Ciclo Lectivo 2026